

API & Authentication

What Is an API?

API stands for **A**pplication **P**rogramming **I**nterface.

It is an interface that allows communication and data exchange between different applications or systems.

An API acts as a **middle layer (mediator)** between the **Front-End** and the **Back-End**, enabling them to communicate with each other smoothly and securely

Why It exists?

APIs solve the problem of connecting different systems and applications in a secure way without needing direct access to the internal code or database, so they make development faster, more organized, secure, and easier to maintain.

APIs advantages:

- **Security:** Protects the database and system from direct user access.
- **Organization:** Provides clear communication rules between systems.
- **Reusability:** One API can be used across multiple platforms.
- **Faster Development:** Front-End and Back-End developers can work independently.
- **Easy Maintenance:** Back-End changes usually do not affect the Front-End if the API stays the same.

How does The Front-End communicates with the Back-End through an API?

The process like this:

1. The user do an action on the interface, such as login or request some data.
2. The Front-End sends a **request** to the API using HTTP over the internet.
3. The API transfers the request to the Back-End.
4. The Back-End processes the request and communicates with the database if needed.
5. The Back-End sends back a **response** with the required data or result.
6. The API returns the response to the Front-End, to display it to the user.

Example:

When a user enters their username and password in a login page, the Front-End sends a login request through the API to the Back-End.

The Back-End checks the entered information in the database to verify whether the account is valid or not.

Then, the Back-End sends a response back through the API.

If the login is successful, the API returns the user data and a success message to the Front-End, which then displays the user's account page or dashboard.

- Rest & Endpoints

REST stands for **R**epresentational **S**tate **T**ransfer

is an architectural style used to design APIs and enable communication between clients and servers through HTTP methods such as GET, POST, PUT, and DELETE.

- REST is:
 - Organizes communication between systems in a simple and standard way.
 - Makes APIs easier to understand and use.
 - Allows frontend and backend applications to communicate efficiently.
 - Supports scalability and flexibility in modern applications.
 - Works with different technologies and programming languages.

Endpoint

An endpoint is a specific URL in an API that represents a resource or a specific function provided by the server. It is the address where a client sends requests to access, create, update, or delete data in a system.

Each endpoint is associated with a particular HTTP method (such as GET, POST, PUT, or DELETE), which defines the type of operation performed on the resource.

- **GET** is used to retrieve or read data from the server without making any changes.
- **POST** is used to create new data or resources in the system.
- **PUT** is used to update or modify existing data.
- **DELETE** is used to remove or delete data from the system.

Request & Response

A **Request** is a message sent from the client (like a frontend application) to the server to ask for data or perform an action. It includes information such as the HTTP method (GET, POST, PUT, DELETE), the endpoint (URL), headers, and optionally a body containing data.

A **Response** is the message returned from the server after processing the request. It includes a status code (such as 200 for success or 404 for not found), the requested data, and sometimes a message describing the result.

=The request is what the client sends, and the response is what the server returns.

Example

For Request:

```
GET /users/1
```

Response:

Status: 200 OK

```
{  
  "id": 1,  
  "name": "Shahed",  
  "email": "shahed@gmail.com"  
}
```

Authentication & Authorization

Authentication is the process of verifying the identity of a user.

It answers the question: “Who are you?” It usually happens during login.

Authorization is the process of determining what an authenticated user is allowed to do. It answers the question: “What are your permissions?” It controls access to specific resources or actions based on user roles.

Basic Authentication (Basic Auth)

Basic Authentication is a simple login method where the user sends their username and password with every request.

Disadvantages:

- Weak security (Base64 is not encryption)
- Credentials are sent with every request
- Not suitable for modern or large-scale applications

Advantages:

- Simple systems -Internal or closed networks
- Basic testing environments

Token-Based Authentication

Token-based authentication is a modern approach where the user logs in once, and the server returns a token that is used for future requests instead of sending credentials every time.

After a user logs in, the server verifies their credentials and generates a token containing user information, which is signed with a secret key. This token is sent back to the client and stored locally. For every subsequent request, the client includes the token in the Authorization header instead of sending login credentials again.

When the server receives a request, it extracts and verifies the token by checking its signature and validity. If the token is valid and not expired, the server grants access to the requested resource; otherwise, it rejects the request with an unauthorized error.

JWT

is a secure and compact token format used for authentication in web applications and APIs. It allows users to remain logged in without sending their credentials with every request.

A JWT consists of three parts:

a **header** (metadata about the token), a **payload** (user data such as user ID or role), and a **signature** (used to verify the token's integrity and ensure it has not been modified).

After a user logs in, the server generates a JWT and sends it to the client. The client stores the token and includes it in the Authorization header for future requests. The server then verifies the token's signature and validity before granting access to protected resources.

=JWT is a stateless authentication method that securely transmits user information.

Password hashing

Password hashing is the process of converting a plain text password into an irreversible fixed-length string using a cryptographic algorithm. The original password is never stored in the database, by storing only encrypted representations of passwords instead of plain text, significantly improving system security.

When a user logs in, the system hashes the entered password and compares it with the stored hash to verify identity.

Why we use Password Hashing

1. Protects user data in case of database breaches
2. Prevents storing plain text passwords
3. Protects users who reuse passwords
4. Limits internal access risks
5. One-way security (irreversible)

Password hashing solves the problem of storing passwords in plain text, which can be easily stolen if a database is hacked.

stores passwords as irreversible hashed values, making them unreadable and protecting user accounts even in case of data breaches.

إعداد: شهد إبراهيم العبدروس

التاريخ: 25/5/2026